Preface

I spent two decades in a corner of finance where software is not allowed to be approximately right.

Fund administration is the plumbing under the asset management industry: net asset values calculated to the fourth decimal, transfer agency, depositary oversight, regulatory reporting to authorities who ask precise questions and expect precise answers. It is not glamorous. What it is, is a place where you develop an unusual relationship with the word *guarantee*. In that world, a system that is right ninety-nine percent of the time is not a good system with a small flaw. It is a system that will, on a schedule you cannot predict, produce a number that goes into a document that goes to a regulator, and somebody will have to explain it.

So when models arrived that could decide things, I had the reaction I suspect a lot of people in regulated industries had. Not *this will never work*, which was obviously wrong, and not *this changes everything*, which was true but useless.

Something narrower and more uncomfortable: **if the software is no longer doing what somebody wrote, where did the guarantees go?**

That question is this book.

What I found, and why it took a book

The short version is that the guarantees do not go anywhere. They stop arriving for free.

Classical software gives you an enormous amount you never asked for and never notice. Control flow you can read. A failure you can reproduce. Behavior that cannot exceed what the code says, because the code is all there is. None of that is a feature anyone designed; it is a side effect of the fact that a human wrote every branch in advance. Put a model in the loop and every one of those free gifts is silently withdrawn, and what replaces them, if you do nothing, is a system that is impressive in a demo and unaccountable in production.

The work, then, is to rebuild deliberately what you used to get by accident. That turns out to be a real engineering discipline with real machinery: state that tells the truth about a world that moves, doorways narrow enough that only well-formed actions get through, receipts, gates on the things that cannot be undone, memory

policies, stopping conditions, verification ladders, and an honest meter. None of it is exotic. Almost all of it will feel familiar to anyone who has built distributed systems, which is the most encouraging thing I can tell you: this is not a new kind of engineering, it is engineering you already know, applied at a boundary that is new.

It took a book because the machinery only makes sense as a whole. Any one piece looks like overhead until you have watched the failure it prevents.

How this book is built

Thirteen chapters, each with three things.

A true story at the front. Not decoration. Every principle here was learned expensively by somebody, usually long before anyone thought about AI, and the story is the shortest path to why the principle exists. The alarm that almost ended the first Moon landing. A secretary in 1966 who asked to be left alone with a computer program. Two spacecraft teams and two systems of units. A loom in Lyon. A man who could not form new memories. Forty-five minutes in 2012 that cost four hundred and forty million dollars. An airline that argued in court that its chatbot was a separate legal entity. None of these are invented and none are decorative; each one is the chapter's argument, already made, by reality.

A lab that runs. Every chapter ends in code you execute, and the output printed in the book is the output the code actually produced. The labs need no API key and no third-party libraries. That is a deliberate design choice, not a limitation: the model in these labs is a deterministic stand-in behind the same interface a real model uses, which means the behavior on the page is the behavior on your machine, and the whole book becomes reproducible. When you want the real thing, Appendix A shows you the five lines that swap it in.

Two closing instructions. One practical, usually asking you to break what you just built, because you learn more from the failure than the success. One that asks you to circle something in pencil, because a few of these ideas do not pay off until much later, and I would rather plant them than pretend the book is finished when it is not.

There is a running example throughout: a company called Meridian with a churn problem. It is invented. Its problem is not, and you will build one system for it, from a deterministic loop in Chapter 1 to a metered, verified, crash-survivable harness by Chapter 13.

What this book is not

It is not a framework tutorial. No framework is taught here, deliberately, and by the end you will understand what all of them are doing underneath, which ages considerably better.

It is not a machine learning book. You will not train anything. The model is treated as a component with known failure characteristics, which is the correct posture for someone who has to ship a system around one.

It is not a book about the future of work. There is a real argument to be made about what happens to organizations when this technology matures, and I intend to make it, but not here and not mixed in with the engineering. This book stops where the engineering stops.

A word on the series, and on honesty

This is the first of four books, and I want to be straightforward about that rather than coy.

The argument that runs underneath all four is simple to state: software evolves by taking things that used to be written in advance and letting the running system produce them instead. This book does that to exactly one layer, the contents of a step, and holds everything else still, so you can see clearly what melting one layer costs and what has to be built to pay for it. Book II does it to the actor, Book III to the flow, and Book IV asks what is left.

But Book I has to stand on its own, and I have tried hard to make it do that. If you never read another word of the series, you should finish this book able to build and defend a production agent, which is the promise, and the promise is not contingent on anything I write later.

I will also say plainly what I am not certain about. Nobody, including me, knows how far this goes or how fast. The engineering in these pages I am confident about, because it is mostly old engineering wearing new clothes and because every lab here runs. The larger arc is an argument, offered as an argument, and you should feel free to take the machinery and leave the philosophy.

One practical note. Every term the series relies on, from artifact and emission to the frozen core, is defined in this book at first use, and all of them are collected in one glossary, Appendix E of Book IV.

Who I hope reads it

Engineers who have built an agent that worked beautifully in a demo and then did something inexplicable on a Tuesday. Architects who have to sign their name under a system whose behavior nobody wrote. Anyone in a regulated or consequential industry who is being asked whether this technology can be trusted, and who wants a better answer than a feeling.

That last group is where I started, and it is who I was writing for when the writing got difficult.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, emphasis, filenames, and the titles of works.

`Constant width`
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, data types, environment variables, statements, and keywords.

Bold
: Marks a term at the moment it is defined, and the occasional sentence the argument turns on.

Shaded boxes hold sidebars: a distinction, caveat, or piece of history that stands beside the argument rather than inside it. Each chapter opens with an epigraph and a true story, ends with a lab, and closes with two instructions, one practical and one in pencil, followed by a short summary under the heading “Where We’ve Landed.”

Using Code Examples

Every chapter ends with a lab, and every lab runs. The thirteen labs and the reference runtime, `meridian_runtime.py`, accompany the manuscript as runnable files, and they are reproduced in full in this book: the runtime in Appendix A, the labs in Appendix I. None of them needs an API key or a third-party package; Python 3.10 or newer is all that is required. The only optional dependency is the `anthropic` package, used solely by the runtime's `--live` flag.

```
./run_all_labs.sh                              # every lab plus the runtime
python3 labs/ch09_the_exoskeleton_lab.py       # any chapter's lab
python3 meridian_runtime.py                    # the reference runtime
python3 meridian_runtime.py --crash-after 2    # kill mid-run, then resume
python3 meridian_runtime.py --live             # needs ANTHROPIC_API_KEY
```

The experiment to run first is `--crash-after 2`. Watch the world counters before and after the crash. They do not change. That is the whole book in two lines of output.

Thanks

To the engineers whose expensive lessons I have borrowed throughout, most of whom never knew they were writing a chapter of somebody's book about AI. And to the reader who checks the labs actually run. You are the reader I wrote them for.

Now: Chapter 1, and a computer alarm, twelve hundred meters above the Moon.